
Predicción de abandono de clientes

PyCaret (AutoML) vs. regresión logística manual en scikit-learn

Asignatura: Ciencia de Datos
David Céleri · José Solórzano

Dataset: Telco Customer Churn (IBM)

¿Qué queríamos responder?

¿Puede PyCaret igualar o superar el desempeño de una regresión logística hecha a mano en scikit-learn al predecir el abandono de clientes, y con qué costo en tiempo y esfuerzo de programación?

Criterios de comparación

Criterio	Cómo se mide
1. Desempeño	AUC-ROC, F1 y recall de la clase «abandona», misma partición y validación cruzada
2. Tiempo	Segundos de entrenamiento y selección de modelo
3. Esfuerzo	Líneas de código necesarias para llegar al modelo
4. Cualitativo	Interpretabilidad y control sobre el proceso

Datos e importación

7.043

clientes

21

columnas

26,5 %

abandona (clase
desbalanceada)

< 1 MB

archivo CSV

Origen

- Telco Customer Churn (IBM Sample Data), publicado en Kaggle.
- Variables: datos demográficos, servicios contratados (teléfono, internet, soporte, streaming), contrato, método de pago y cargos.
- Objetivo: **Churn**, si el cliente abandonó la empresa en el último mes.

Descarga automática

- Se descarga con **kagglehub** dentro del notebook, sin pasos manuales.
- Respaldo: si Kaggle falla, se usa la copia que IBM publica en GitHub.
- Se guarda en data/; si ya existe no se vuelve a descargar.

Revisión de calidad y limpieza

Lo que encontramos

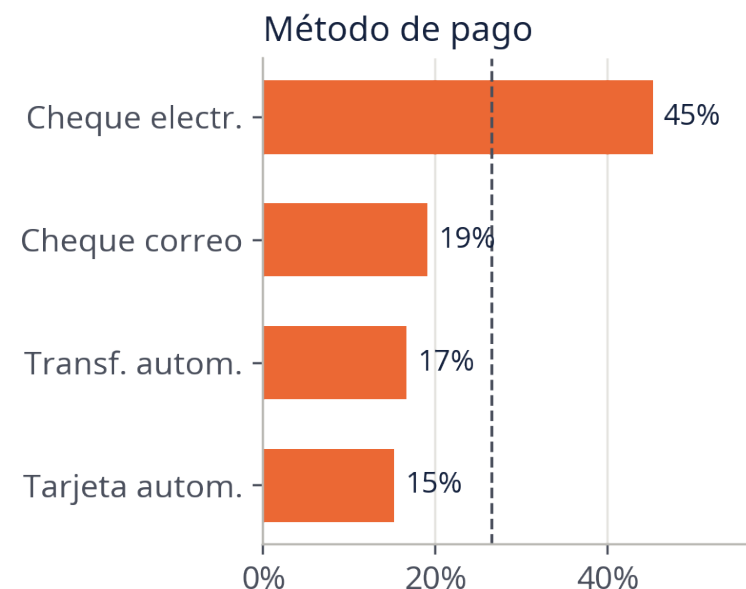
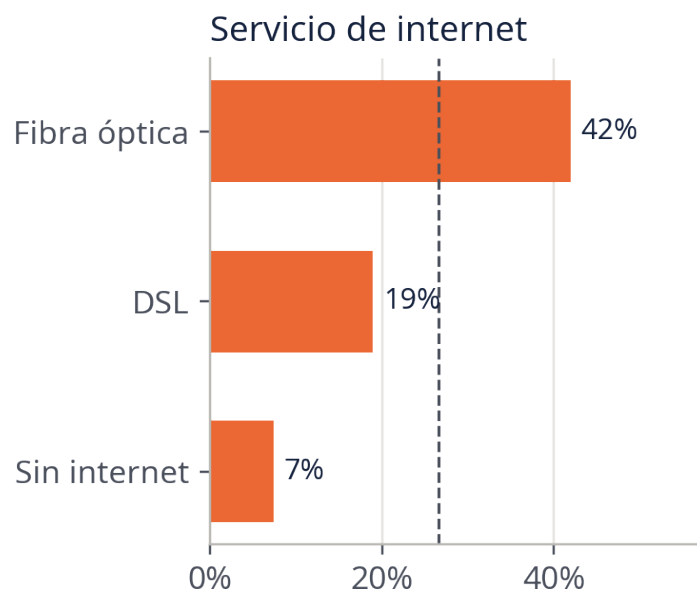
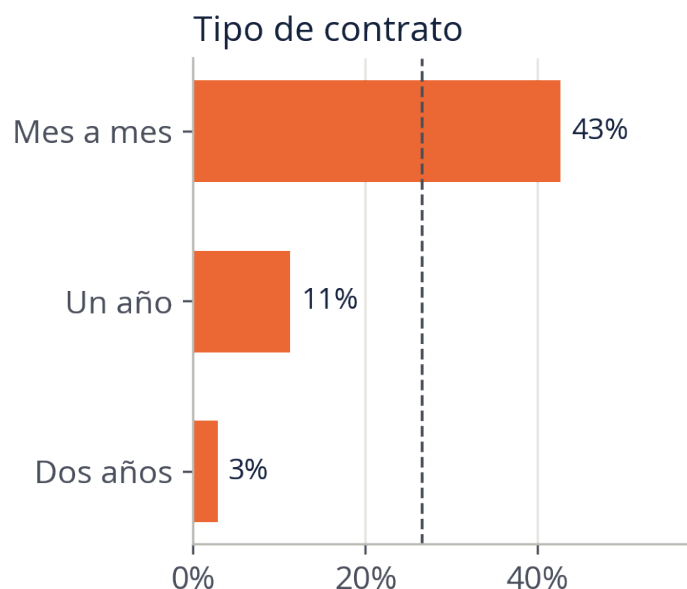
- Sin valores nulos ni registros duplicados.
- **TotalCharges** venía como texto: 11 celdas en blanco.
- Esas 11 filas tienen **tenure = 0**: clientes nuevos que aún no pagaban.
- **SeniorCitizen** codificada como 0/1, distinta al resto (Yes/No).

Lo que hicimos

Acción	Motivo
TotalCharges a numérica; vacíos = 0	Es su valor real, no un dato perdido
Eliminar customerID	Identificador sin valor predictivo
SeniorCitizen a Yes/No	Consistencia con las demás categóricas
Churn a 1 / 0	Misma clase positiva en ambos enfoques

Resultado: **data/telco_churn_clean.csv** · 7.043 filas × 20 columnas · 0 faltantes. La codificación y el escalado se dejaron a cada enfoque para que ambos partan del mismo archivo.

Análisis exploratorio: quién abandona



Línea punteada = tasa global de abandono (26,5 %)

- Los que se van son clientes nuevos: mediana de **10 meses** de antigüedad frente a **38** de los que se quedan.
- Género y tener servicio de teléfono casi no influyen. Clases desbalanceadas: se usa AUC y partición estratificada.

Mismas condiciones para ambos modelos

Partición 80 / 20

Estratificada: 5.634 clientes para entrenar y 1.409 para prueba, ambos con 26,5 % de abandono.

Partición compartida

Los índices se guardan en `particion_train_test.csv` y scikit-learn usa exactamente las mismas filas.

Validación cruzada

5 folds estratificados sobre el conjunto de entrenamiento.

Semilla fija

`random_state = 42` en todo el proceso: resultados reproducibles.

Así cualquier diferencia en las métricas se debe al modelo y no a los datos usados.

Enfoque 1: PyCaret (AutoML)

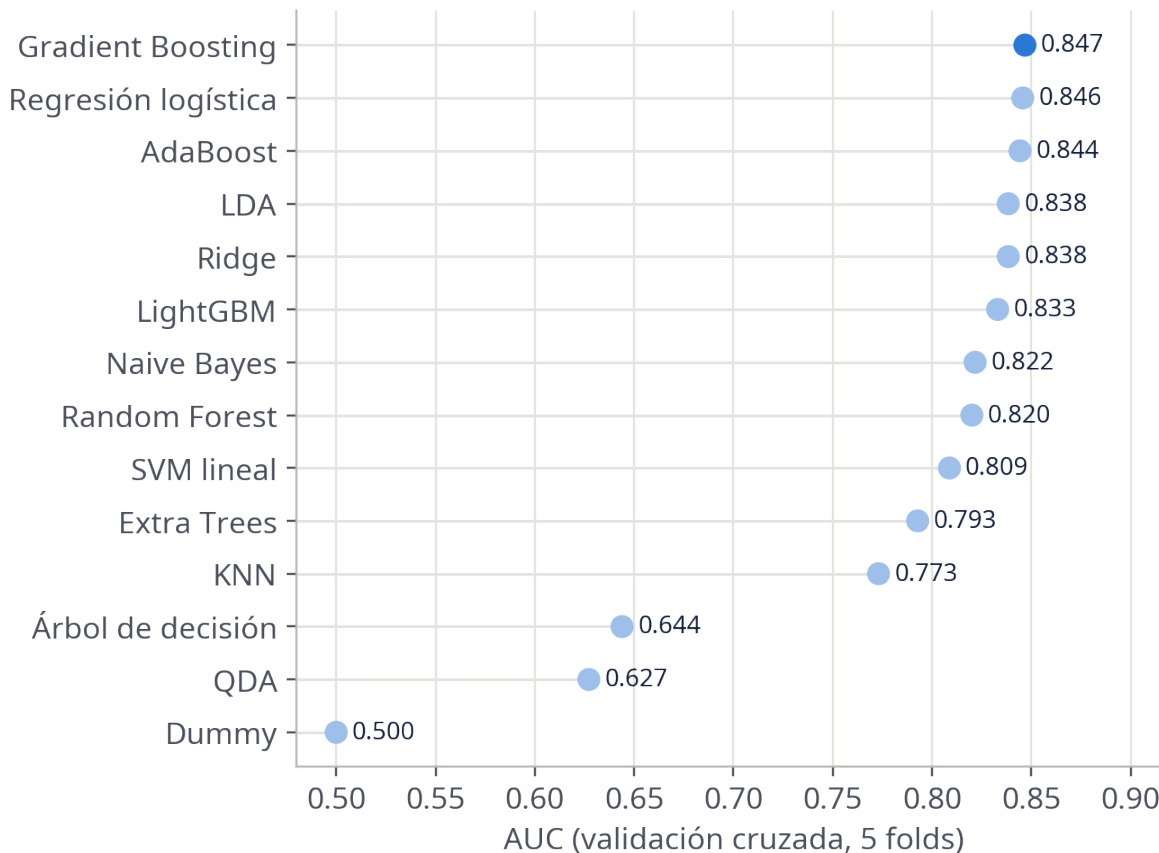
Configuración con `setup()`

- Imputación, codificación (binarias a 0/1, resto one-hot) y escalado z-score automáticos.
- Sin balanceo ni selección de variables, para comparar en igualdad.
- `compare_models()` entrenó **14 algoritmos** en ~ 14 s, ordenados por AUC.

Mejor modelo: **Gradient Boosting**

AUC 0,847 en validación cruzada y 0,843 en prueba. La regresión logística quedó segunda (0,846): los tres primeros están prácticamente empatados.

AUC por modelo (validación cruzada)



Enfoque 2: regresión logística en scikit-learn

Pipeline escrito a mano

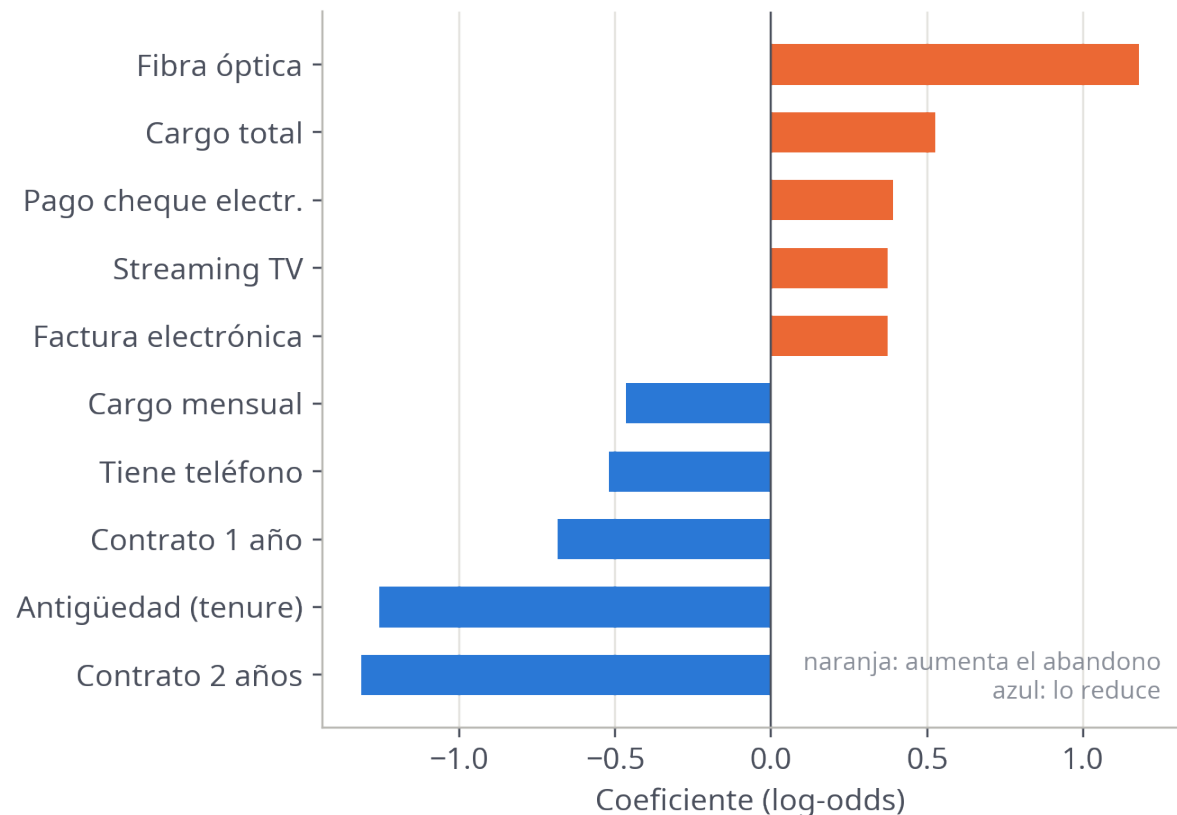
- ColumnTransformer: StandardScaler para numéricas y OneHotEncoder para categóricas.
- LogisticRegression con parámetros por defecto.
- Misma partición y validación cruzada de 5 folds.

Resultado en prueba

AUC 0,842 · F1 0,605 · Recall 0,559

Ventaja: cada coeficiente explica cómo influye una variable en el riesgo de abandono.

10 coeficientes más influyentes



Comparación con márgenes de error

Una diferencia solo es real si su intervalo de confianza del 95 % **no incluye el 0**.

Validación cruzada repetida

5 folds × 10 repeticiones = 50 evaluaciones por modelo, con los mismos folds para ambos. Se usa el t-test corregido de Nadeau y Bengio porque los folds comparten datos.

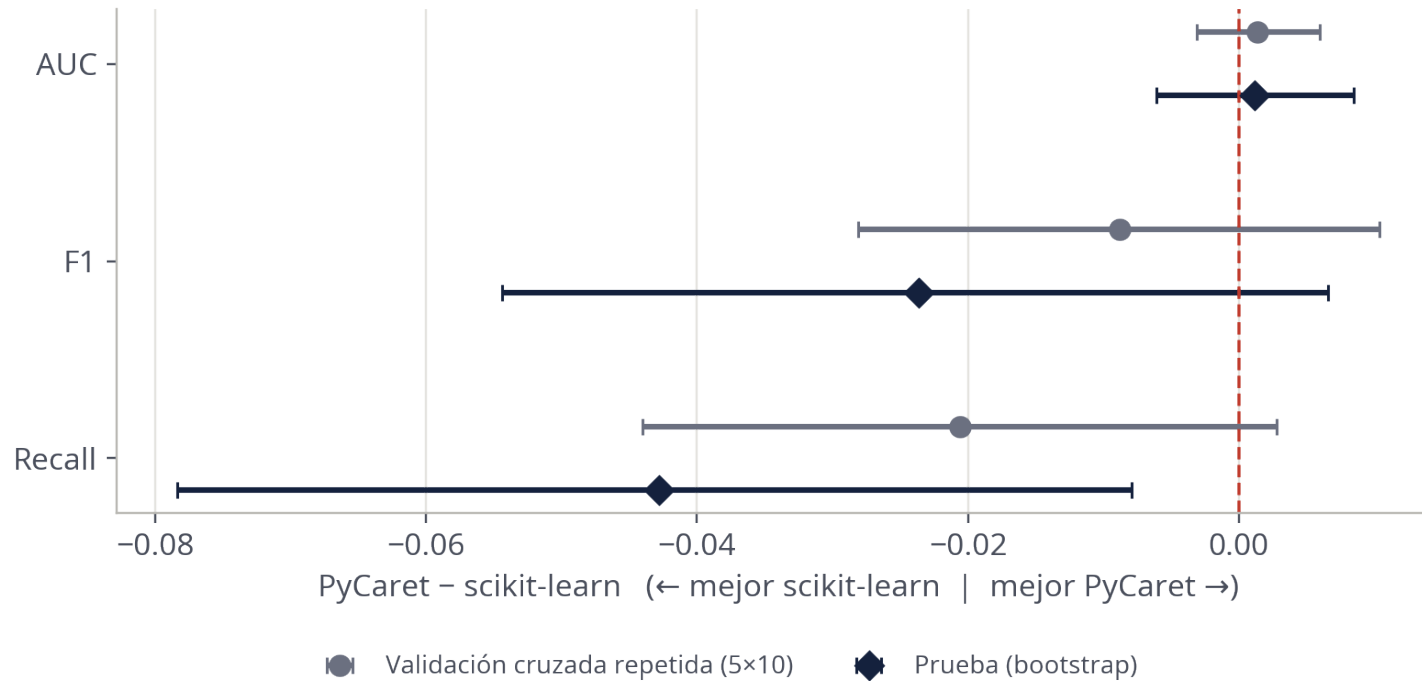
Bootstrap sobre prueba

2.000 remuestreos de los 1.409 clientes de prueba, evaluando los dos modelos sobre los mismos clientes en cada remuestreo.

Métricas obtenidas (prueba, umbral 0,5)

Modelo	AUC	F1	Recall
PyCaret · Gradient Boosting	0,843 [0,820 – 0,865]	0,581 [0,534 – 0,623]	0,516 [0,466 – 0,565]
scikit-learn · Reg. logística	0,842 [0,819 – 0,864]	0,605 [0,558 – 0,646]	0,559 [0,505 – 0,609]

Resultado: no hay diferencia demostrable



AUC

Diferencia de 0,001. Empate.

F1

Ambos intervalos incluyen el 0. Empate.

Recall

Solo en prueba favorece a la regresión logística; en validación cruzada no se confirma y depende del umbral 0,5. No concluyente.

Si el intervalo cruza la línea roja, no se puede decir que un modelo sea mejor.

Tiempo y esfuerzo de programación

	PyCaret	scikit-learn
Líneas de código (código mínimo, con imports)	5	17
Modelos evaluados	14	1
Tiempo hasta el modelo	~ 16 s	~ 1,5 s
Interpretación	Importancia global de variables	Coeficientes con dirección y magnitud
Control del preprocesamiento	Automático (hay que revisarlo)	Explícito, paso a paso
Entorno	Restringido: requiere Python 3.11	Ligero y actualizado

PyCaret ahorra código y explora muchos modelos con una línea, a cambio de ~ 10 veces más tiempo de cómputo y menos control sobre lo que hace.

Conclusiones

Desempeño	Sin diferencia real: en AUC (0,847 vs. 0,846) y F1 los intervalos incluyen el 0.
Recall	Leve ventaja de la regresión logística en prueba, no confirmada en validación cruzada.
Esfuerzo	PyCaret: 5 líneas vs. 17 y 14 modelos evaluados, pero ~ 10 veces más lento.
Respuesta	PyCaret igual a la regresión logística manual, pero no la supera .
Recomendación	Usar la regresión logística: rinde igual, es más rápida e interpretable.

Limitaciones: hiperparámetros por defecto, umbral fijo de 0,5 y sin corrección por comparaciones múltiples.